Late Breaking Results: Scalable GPU-Friendly Parallelization for Sweep-Based Maze Routing

Cheng-Yu Chiang[1], Zong-Ying Cai[2], Chao-Chi Lan[2], Yan-Jen Chen[1], Yang Hsu[1], Yao-Wen Chang[1,2], and Hung-Ming Chen[3]
[1]Graduate Institute of Electronics Engineering, National Taiwan University, Taipei 106319, Taiwan
[2]Department of Electrical Engineering, National Taiwan University, Taipei 106319, Taiwan
[3]Institute of Electronics and SoC Research Center, National Yang Ming Chiao Tung University, Hsinchu 30010, Taiwan
frankchiang@eda.ee.ntu.edu.tw; zycai@eda.ee.ntu.edu.tw; b09901030@ntu.edu.tw; yjchen@eda.ee.ntu.edu.tw;
yanghsu@eda.ee.ntu.edu.tw; ywchang@ntu.edu.tw; hmchen@nycu.edu.tw

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11132843

***Abstract*—Global routing is a critical stage in the VLSI design flow, aiming to provide a robust guide for detailed routing and serve as early design feedback for placement. Many approaches have leveraged GPU parallelization to achieve significant acceleration. However, with the fast-growing complexity of modern large-scale designs, recent GPU-accelerated maze routing algorithms, driven by the *sweep* operation, struggle to find solutions efficiently with limited GPU memory resources. In order to address this issue, this paper proposes a scalable, GPU-friendly sweep-based maze routing that requires significantly less memory and fewer kernel function calls while accelerating overall runtime. We introduce a sweep-sharing technique that allows multiple nets to be routed simultaneously within a single sweeping process, substantially reducing memory consumption and kernel launching overhead. We further propose an edge-level rip-up-and-reroute technique that selectively reroutes only overflowed segments, preserving feasible parts of the solution to reduce runtime substantially. Experimental results on the latest ISPD'24 Contest benchmarks demonstrate that our GPU-friendly maze routing with sweep sharing can significantly improve the efficiency of the state-of-the-art GPU-accelerated maze router.**

I. Introduction

Global routing plays a pivotal role in the VLSI design flow, providing robust routing guides for detailed routing while offering early feedback on wirelength, congestion, and routability during the placement and planning stages. It enables efficient optimization of routing resources by estimating key metrics early in the design process, helping to reduce potential design rule violations and improve overall circuit performance. Global routing typically involves two stages: pattern routing and maze routing. Pattern routing employs predefined patterns, such as L-shapes or Z-shapes, to efficiently generate initial paths, followed by maze routing, which reroutes nets crossing overflowed grids to explore alternative paths. Maze routing focuses on solving the multi-source-multi-target shortest path problem on a grid graph. However, this step is often highly time-consuming due to the extensive computational resource demands for handling large grid graphs, as maze routing involves complex path exploration and cost updates. Traditional methods [1], [2], [3] have attempted to address this issue through techniques such as limiting search space and applying rip-up-and-reroute strategies to reduce computational overhead.

To address scalability challenges in modern designs, recent studies [4], [5], [6] have utilized parallelization techniques on graphics processing units (GPUs) to accelerate the global routing stage. Methods like the GAMER [4] have demonstrated significant speedups on maze routing with *sweep* operation, which replaces cost accumulation with prefix sum accelerated by GPU parallelization.

A. Motivation

However, in recent GPU-accelerated maze routing work [4], performance is significantly hindered by two major issues, as shown in Figure 1. First, GAMER invokes a large number of GPU kernel function calls of *sweep* operations for nets. Each kernel launch incurs a non-negligible waiting time, leading to substantial runtime overhead when processing large numbers of nets. Our method addresses this by introducing a sweep-sharing strategy, allowing multiple nets to utilize a single sweeping process. This strategy drastically reduces kernel function calls and improves runtime efficiency.

Second, GAMER allocates a separate cost map with a full layout size for each net, which consumes excessive GPU memory and limits parallelism due to memory constraints. In contrast, our approach only requires a single global cost map shared across all nets, thereby enhancing memory utilization and enabling higher degrees of parallelism in large-scale designs. These optimizations are critical for achieving both scalability and efficiency in modern global routing applications.

B. Problem Formulation

We formulate our problem as follows:

Problem 1 (The Maze Routing Problem): Given a netlist, routing resource, and pattern routing result, determine the routing result for each net while minimizing the total wirelength, via count, overflow cost, and runtime.

This work was partially supported by AnaGlobe, ASUS, Delta Electronics, Google, Maxeda Technology, TSMC, NSTC of Taiwan under Grant NSTC 110-2221-E-002-177-MY3, NSTC 113-2218-E-002-041,NSTC 113-2223-E-002-007, and NSTC 113-2640-E-002-001.

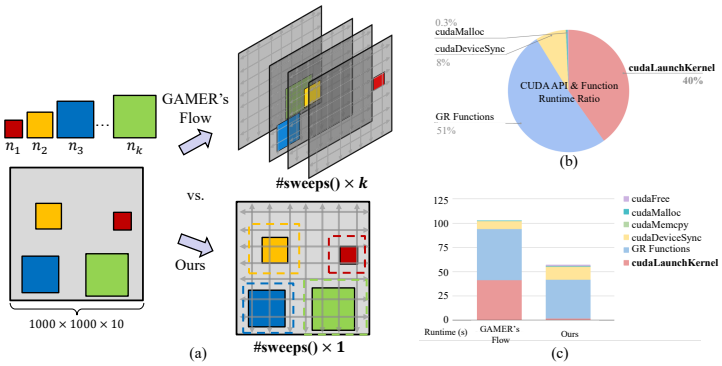

Fig. 1. A comparison of sweep-based maze routing strategies with a k-net case study. (a) Illustration of the difference between two parallelization strategies. (b) Runtime ratio of CUDA APIs and function. (c) Comparison of runtime and its component breakdown. GAMER's flow allocates k layout spaces and runs k sets of *sweep* operations for the k nets. Instead, our flow only allocates one layout space and one set of *sweep* operations for the k nets. This strategy significantly saves the runtime from the delays caused by GPU kernel launches, which often comprise around half of the total runtime, by reducing the function calls of *sweep* operations. Additionally, our flow allows more nets to be routed simultaneously, which would otherwise be constrained by GPU memory limitations.

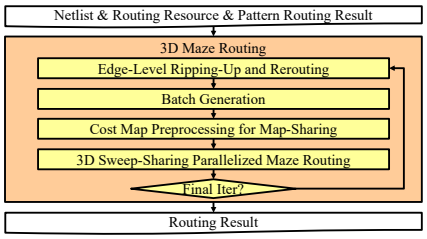

Fig. 2. The overview of our proposed flow.

II. Methodology

In this section, we first give an overview of our framework, which consists of four major stages: (1) *Edge-Level Ripping-Up and Rerouting*, (2) *Batch Generation*, (3) *Cost Map Preprocessing for Map-Sharing*, and (4) *3D Sweep-Sharing Parallelized Maze Routing* and then detail our methods. In *Edge-Level Ripping-Up and Rerouting*, we first break the net edges that pass through the overflowed grids and reroute them in the following stages. In *Batch Generation*, we group nets that can be independently routed into a batch and perform our routing algorithm sequentially on these batches. In *Cost Map Preprocessing for Map-Sharing*, we prepare a cost map to facilitate the map-sharing technique in the sweep-based maze routing algorithm. In *3D Sweep-Sharing Parallelized Maze Routing*, we perform sweep-based maze routing with sweep-sharing to connect all pins of each net.

A. Edge-Level Ripping-Up and Rerouting

Instead of ripping-up the whole overflowed nets, we rip-up only the affected net edges to eliminate overflowed regions, allowing subsequent stages to efficiently reroute these edges without disturbing the rest of the solution from the pattern route. Overflowed nets are ripped-up into multiple connected components and rerouted. Each connected component is transformed into a pin, with its nodes and remaining unripped routes converted into access points of the pin to be used in the subsequent routing stages.

B. Batch Generation

After ripping-up overflow nets, this stage focuses on maximizing routing efficiency by grouping independent nets with non-overlapping net boundaries into a batch, allowing nets within the same batch to be routed simultaneously without interference. The subsequent routing algorithm then parallelly routes the nets for each batch with GPU.

TABLE I

THE STATISTICS OF THE BENCHMARKS AND COMPARISONS OF PERFORMANCE FOR GPU-ACCELERATED MAZE ROUTING.

Case ID	Benchmark	#Nets	Gcell Grid	Ripped-Up #Nets	GAMER Score	GAMER Runtime (s)	GAMER *cudaLK* Total Time (s)	GAMER *cudaLK* Instances	GAMER #Batch	GAMER Max Batch Size	GAMER Max GPU MEM Usage (GB)	Ours Score	Ours Runtime (s)	Ours *cudaLK* Total Time (s)	Ours *cudaLK* Instances	Ours #Batch	Ours Max Batch Size	Ours Max GPU MEM Usage (GB)
0	ispd18_test5	72K	619x613x9	31,522	2.09E+07	56.33	65.75	3475012	529	239	38.25	2.14E+07	17.31	3.53	792598	544	146	4.87
1	ispd18_test8	180K	905x883x9	79,160	4.73E+07	166.27	196.94	8669981	830	503	37.98	4.75E+07	51.51	7.30	1644132	825	242	11.56
2	ispd18_test10	182K	606x522x9	91,888	4.73E+07	161.44	163.08	9273107	2346	605	31.67	4.68E+07	74.87	8.50	1841680	2173	1221	13.13
3	ispd19_test7	359K	1053x1011x9	27,735	7.49E+07	145.05	165.41	7718208	1784	143	35.46	7.42E+07	34.59	6.82	1636815	1693	94	4.58
4	ispd19_test8	538K	1202x1138x9	161,377	2.62E+08	573.51	604.54	22315923	1483	265	35.91	2.62E+08	175.37	17.04	3511419	1283	624	23.05
5	ispd19_test9	895K	1337x1433x9	282,807	1.93E+08	1190.99	1290.15	39265632	2044	181	36.16	1.93E+08	432.77	28.47	5249469	1293	1889	39.82
6	ispd18_test5_metal5	72K	619x613x5	32,622	1.79E+07	48.83	53.87	3508729	532	204	34.38	1.77E+07	16.44	3.45	794626	544	136	4.96
7	ispd18_test8_metal5	180K	905x883x5	80,569	4.08E+07	130.42	157.32	8712045	835	485	33.70	4.07E+07	51.35	7.11	1649351	825	241	11.63
8	ispd18_test10_metal5	182K	606x522x5	92,089	4.39E+07	143.53	155.53	9214411	2250	747	24.94	4.37E+07	67.59	8.19	1791785	2111	1178	13.11
9	ispd19_test7_metal5	359K	1053x1011x5	30,994	6.74E+07	105.07	125.27	7933093	1669	218	30.06	6.70E+07	28.47	6.69	1541342	1691	115	4.87
10	ispd19_test8_metal5	538K	1202x1138x5	163,926	2.54E+08	402.28	455.77	22646740	1397	438	30.15	2.53E+08	146.79	16.55	3452508	1272	637	23.19
11	ispd19_test9_metal5	895K	1337x1433x5	285,580	1.90E+08	779.75	895.33	39357725	1273	293	30.02	1.89E+08	338.89	27.11	5214494	1256	1561	39.90
Avg. Comp.					1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	0.36	0.04	0.18	0.94	2.22	0.50

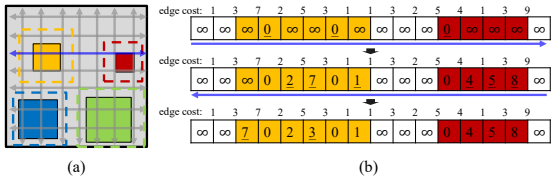

Fig. 3. An illustration of the sweep-sharing strategy on costs accumulating with *sweep* operation. (a) *sweep* operations on the entire layout. (b) Detailed process for a *sweep* operation.

C. Cost Map Preprocessing for Map-Sharing

Before the cost calculation and path finding, we prepare a global cost map structure that enables our routing algorithm to calculate costs for multiple nets on a single cost map during the sweep-based maze routing process. We mark each grid in a row or column according to the boundary of its corresponding net, ensuring that cost calculations are restricted within each group. To avoid overly restricting the solution space, we apply padding to expand the net boundaries as indicated by the dashed lines in Figure 1, allowing nets to be routed beyond their original boundaries.

D. 3D Sweep-Sharing Parallelized Maze Routing

In this stage, we apply the sweep-based maze routing algorithm with our sweep-sharing technique to perform parallelized maze routing across 3D layers, ensuring all pins of each net are connected with minimized total wirelength, via count, and grid overflow. The sweep-based maze routing algorithm consists of two main stages: (1) cost calculation and (2) shortest path finding.

In the cost calculation stage, the *sweep* operation accumulates costs and updates the shortest distances to all grids with edges in one direction utilizing prefix sum accelerated by GPU parallelization. By alternating horizontal and vertical sweeps many times, the shortest distance to every grid can be found. With the marked cost map, we only update the costs from the grids marked as the same group. This approach restricts cost updates within each net boundary, allowing a single sweep operation to update costs within multiple net boundaries at the same time.

In the shortest path finding stage, based on the updated cost map, the maze routing algorithm starts from the unconnected pin with the lowest cost. It then backtraces through neighboring grids where the costs were updated, forming the shortest path to connect all pins. Figure 4 illustrates an example of how the algorithm works. In each iteration, we first set zero cost to the grids on the starting points, which are the pins and grids marked as routed. Second, we calculate the costs and find an unrouted pin on the grid with the lowest cost. Next, we backtrace, find the shortest path from the pin, and mark the traversed grids as routed. The path-finding stage is performed in parallel for each net and is iteratively called until all pins are connected. Note that this sweep-based maze routing algorithm is a multi-source-multi-target algorithm since the costs are swept out from all traversed grids and the target pins consist of multiple access points.

III. EXPERIMENTAL RESULTS

Our experiments were conducted on a 64-bit Linux machine with AMD EPYC 7313 @ 3.0GHz, one NVIDIA RTX A6000 GPU, and 192 GB RAM. We compared our maze router with the state-of-the-art GPU-accelerated maze router, part of GAMER [4], denoted as GAMER*. The inputs of both routers were obtained from the L-shaped pattern routing results in CUGR 2.0 [3]. We verified our work based on the same benchmark suite, the 2019 CAD contest at ICCAD on Global Routing (ICCAD19) [7], in the GAMER paper [4] and evaluated the results using the state-of-the-art evaluator from the 2024 ISPD Contest on GPU-accelerated Global Routing (ISPD24) [8]. The benchmark statistics are shown in Table I. Finally, the NVIDIA Nsight Systems [9] were used to obtain the CUDA GPU kernel results.

Note that the works [5], [6] experimented on the 2024 ISPD benchmarks, but they addressed a different problem on global routing while this paper is on maze routing; as a result, we did not compare with these works.

As shown in Table I, our method achieves an average 2.78x speedup with less than 1% variation in score compared to GAMER*'s flow. The speedup mainly benefits from the reduction in GPU kernel function calls, *cudaLaunchKernel*, which is represented as "*cudaLK*" in the table. As shown in Figure 5, with the proposed sweep-sharing technique, the sweep-based maze routing flow reduces the GPU kernel function calls (red parts) by an average of 82%. Furthermore, by allocating only a single layout-sized cost map in each batch routing, our memory usage is significantly reduced by 50% on average, enabling larger batch sizes in most cases and thereby improving parallelization.

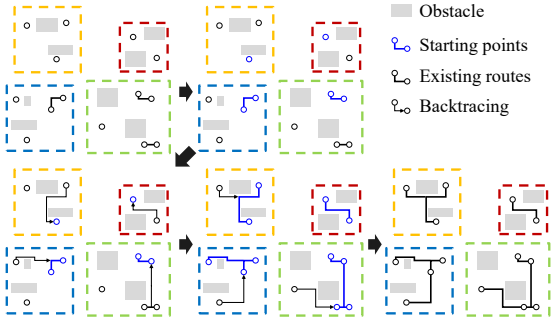

Fig. 4. An illustration of edge-based ripping-up and rerouting with a four-net example.

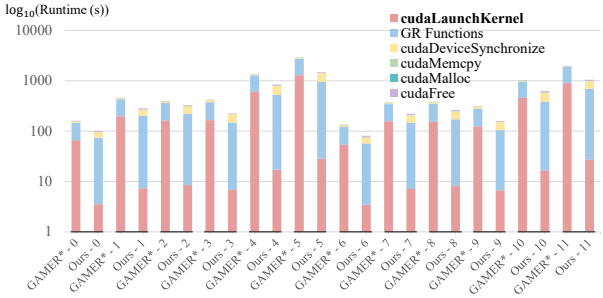

Fig. 5. Comparison of runtime and its component breakdown between GAMER*'s flow and our method for each benchmark.

REFERENCES

[1] W.-H. Liu, W.-C. Kao, Y.-L. Li, and K.-Y. Chao, "NCTU-GR 2.0: Multithreaded collision-aware global routing with bounded-length maze routing," *IEEE Tran. on CAD*, vol. 32, no. 5, pp. 709–722, 2013.

[2] J. Liu, C.-W. Pui, F. Wang, and E. F. Y. Young, "CUGR: Detailed-routability-driven 3D global routing with probabilistic resource model," in *Proc. of DAC*, Virtual, CA, July 2020, pp. 1–6.

[3] J. Liu and E. F. Y. Young, "EDGE: Efficient DAG-based global routing engine," in *Proc. of DAC*, San Francisco, CA, July 2023, pp. 1–6.

[4] S. Lin, J. Liu, E. F. Y. Young, and M. D. F. Wong, "GAMER: GPU-accelerated maze routing," *IEEE Tran. on CAD*, vol. 42, no. 2, pp. 583–593, 2023.

[5] S. Lin, X. Liang, J. Liu, and E. F. Y. Young, "InstantGR: Scalable GPU parallelization for global routing," in *Proc. of ICCAD*, Newark, NJ, October 2024, pp. 1–8.

[6] C. Zhao, Z. Guo, R. Wang, Z. Wen, Y. Liang, and Y. Lin, "HeLEM-GR: Heterogeneous global routing with linearized exponential multiplier method," in *Proc. of ICCAD*, Newark, NJ, October 2024, pp. 1–9.

[7] S. Dolgov, A. Volkov, L. Wang, and B. Xu, "2019 cad contest: LEF/DEF based global routing," in *Proc. of ICCAD*, Westminster, CO, November 2019, pp. 1–4.

[8] R. Liang, A. Agnesina, W.-H. Liu, and H. Ren, "GPU/ML-enhanced large scale global routing contest," in *Proc. of ISPD*, Taipei, Taiwan, March 2024, pp. 269–274.

[9] NVIDIA Corporation, *NVIDIA Nsight Systems*, 2025. [Online]. Available: https://developer.nvidia.com/nsight-systems